Gait-Phase-Aware Command Timing for Decentralized Quadruped Swarms

Bishwaswarup Nayak
Indian Institute of Science, Bengaluru, India
bishwaswarup@iisc.ac.in (corresponding author)

Abstract

Multi-robot collision avoidance treats each robot as a point that can change velocity at any instant. A trotting quadruped cannot: how well it absorbs a change of commanded velocity depends on where in the gait cycle the change arrives. We measure this dependence for a Bézier-curve trot controller on a simulated Unitree Go1 and use it in a decentralized swarm. The phase-aware layer solves reciprocal collision avoidance (ORCA) one gait period ahead, releases each new velocity command at the gait phase our capability map predicts is least destabilizing, and shapes it with a gait-aware slew limiter; urgent conflicts bypass the timing. In MuJoCo, across 4 to 32 robots in antipodal swaps, pillar fields, a narrow gate and a 53 m obstacle course, the full system is compared with the same stack without timing (790 runs, paired by seed, Holm-corrected). In crowded and obstructed scenarios it cuts urgent avoidance events by 22–39% and 99th-percentile body tilt by 19–24%, increases the closest robot–robot distance in formations by 2–5 cm, and had no falls in any run. Prediction without phase timing gives no consistent benefit, so the timing is the active ingredient. In sparse traffic (four robots) the method costs more than it saves.

Keywords: legged robots · multi-robot systems · collision avoidance · gait phase · quadruped swarms · ORCA

1 Introduction

Decentralized collision avoidance methods such as reciprocal velocity obstacles and ORCA [1, 2] give each robot a new velocity every control tick. They assume that the platform can follow that velocity, possibly subject to an acceleration limit. Wheeled robots and multirotors come close to that assumption. A trotting quadruped does not: its body is supported by two diagonal feet at a time, the support line rotates during each stance, and a velocity change that arrives at the wrong moment produces a large body tilt or a stumble.

That a legged gait responds to a disturbance differently at different phases is well known in single-robot locomotion: push recovery, capture-point and step-timing methods all exploit it [3–6]. What has not been studied, to our knowledge, is the consequence for multi-robot navigation. A swarm layer that issues frequent avoidance commands is a source of disturbances to every robot's gait. Formation and swarm controllers for quadrupeds [7, 8] shape *what* each robot is commanded, but not *when* within the gait the command is delivered.

This paper asks one question: does timing the swarm layer's velocity changes to the gait phase make a quadruped swarm safer, and at what cost? We answer it in simulation with a fully open stack and paired statistics. Our contributions are:

1. A measurement of phase-dependent fragility for a Bézier trot on a Unitree Go1 model: a pilot experiment, a capability map over phase, direction and size of a command change (held-out R^2 0.94–0.98, against 0.70–0.87 without phase), and a push-impulse envelope in which knowing the phase raises the tolerable impulse by a factor of 1.33–1.44.

2. A phase-aware command layer: a commander that releases each velocity change at the predicted best phase within a deadline, and a gait-aware slew limiter that shapes the change by phase.

3. A predictive ORCA layer that plans one gait period ahead, so that the commander has time to wait for a good phase, with an urgent bypass when the time to collision is short.

4. An ablation and scaling study over 4–32 robots, a gate, pillar fields and a 53 m obstacle course (790 runs in the final configuration, paired by seed, Holm-corrected), including negative results: where the method does not help, and a robustness fix we tried and rejected.

2 Related work

Reciprocal collision avoidance. Velocity obstacles and their reciprocal forms [1, 2] are the standard decentralized method for crowds of robots; ORCA reduces each control tick to a small linear program. Extensions add platform limits: acceleration-velocity obstacles bound the acceleration [9], and NH-ORCA bounds the tracking error of non-holonomic robots [10]. All of these model the platform's limits as constant in time. We keep ORCA unchanged and instead model a limit that varies periodically with the gait.

Multi-quadruped formation and navigation. Recent work on quadruped teams uses distributed model predictive control with control barrier functions [7] or leader–follower formations with path planning [8]. These treat each quadruped as a velocity-controlled point and rely on the locomotion controller to follow the commands. The phase-aware layer proposed here is complementary: it can sit between any such planner and the locomotion controller.

Phase-dependent response of legged gaits. The capture point [3], step-timing adaptation [4] and switched-system capturability for quadrupeds [5] all show that what a legged robot can recover from depends on its state within the step, and gait-sensitivity analysis [6] measures how disturbances propagate through a limit cycle. These results are used to choose footholds and step times for one robot. We use the same fact one level up: to choose *when* the navigation layer changes its command.

Command shaping for legged robots. Velocity commands to legged controllers are commonly rate-limited, and learned controllers are trained on curricula of command changes. A recent example schedules commands by the robot's current velocity to reach high speed [11]. To our knowledge, no prior work schedules navigation commands by gait phase.

3 The low-level controller and its phase-dependent fragility

Controller. Each robot runs the same open-source trot controller. The feet follow a 12-point Bézier swing curve [12, 13] in a diagonal trot (period 0.36 s, duty factor 0.5, swing clearance 0.07 m, hip-to-foot height 0.27 m). A PI loop on body velocity ($k_i = 1.5$) removes steady drift, Raibert foot placement [14] (gain 0.1) corrects velocity errors, and joint targets go to the position actuators of the MuJoCo Menagerie Go1 model [15, 16]. A slew limiter bounds the rate of change of the command at $(1.5\,\mathrm{m/s^2}, 1.0\,\mathrm{m/s^2}, 3.0\,\mathrm{rad/s^2})$ for (v_x, v_y, ω_z). The controller tracks commands to within about 0.02 m/s, survives 4 min of random command changes without a fall, and walks at heights from 0.16 m to 0.34 m.

Stability. Treating one gait period as a stroboscopic Poincaré map, the largest Floquet multiplier is 0.42–0.59 for the full controller and 0.73–0.87 open loop, so the gait is orbitally stable over the commanded speed range. Stability of the cycle does not say how a single command change is absorbed, which depends on when it arrives.

Why the phase matters. In a trot the body is supported by one diagonal pair at a time: front-right with rear-left, then front-left with rear-right, at roughly $-34°$ and $+34°$ to the heading for the Go1's geometry. Within one stance the effective support axis rotates from about $+25°$ to $+65°$ as the body moves over the feet. A change of commanded velocity that pushes the body across the current support axis is resisted by the stance legs; one that pushes along it tips the body about that axis. The same change therefore produces different tilt at different phases, and by the mirror symmetry of the trot the pattern repeats every half period with the lateral direction flipped.

Measured fragility. A pilot experiment applied command steps and body pushes at controlled phases. Phase had a large effect: in 27 of 64 push conditions, whether the robot recovered depended on the phase alone. From these data we built a capability map: a lookup table, periodic in phase and symmetrized, that predicts peak tilt and fall risk for a command change of a given direction and size at a given phase (Fig. 1a). On held-out trials it explains 94–98% of the variance, against 70–87% for the same model without phase. A push-impulse envelope found by bisection (Fig. 1b) agrees with the pilot in 98.6% of cases, and knowing the phase raises the impulse the robot tolerates by a factor of 1.33–1.44.

What this means for a swarm. In isolated step tests, releasing a command at the best phase strictly dominated the slew limiter alone. In a stream of swarm commands, however, timing alone did not work: only 0–2% of the commands resembled the settled single steps the map was measured on, and about half were urgent, leaving no time to wait for a good phase. Section 4 is designed around that finding: the swarm layer must look ahead so that most changes are known before they are needed.

4 Method

Every robot runs the same four-stage stack at 10 Hz (Fig. 2), with no central planner and no communication beyond the neighbour positions and velocities that ORCA needs. A safety layer

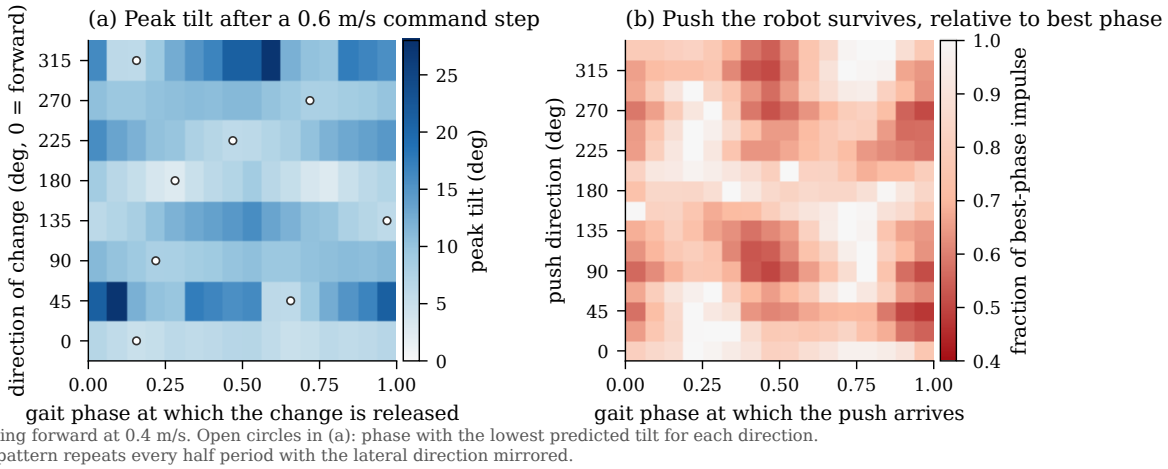


Figure 1: Phase dependence of the trot controller while walking forward at 0.4 m/s. (a) Peak body tilt after a 0.6 m/s step in commanded velocity, by direction of the change and the gait phase at which it is released; open circles mark the best phase for each direction. (b) Largest push impulse the robot survives, as a fraction of the value at its best phase for that direction.

that is identical in every configuration, the baseline included, wraps the stack.

Predictive swarm layer. Each robot is a disc of radius 0.38 m with maximum speed 0.6 m/s. Rather than solving ORCA for the present, the robot solves it for the situation one lookahead L ahead ($L = 0.36$ s, one gait period). Its own position is extrapolated with the command it is currently executing, and its neighbours with their measured velocities. The result is the velocity the robot will need at time $t + L$, so it can be requested with a deadline of L. If the current command would collide within 0.8 s with a robot, or within 1.5 s with a static obstacle, the request is urgent: ORCA is solved for the present and the command is released at once. A 15° keep-right bias breaks symmetric deadlocks.

Phase-aware commander. A request whose change from the current command exceeds 0.15 m/s (or 0.3 rad/s) is a manoeuvre. For each manoeuvre the commander queries the capability map for every gait phase reachable before the deadline (at most 0.5 s) and releases the command at the phase with the lowest predicted cost, namely peak tilt plus a weight of 50 on the predicted fall probability. Smaller changes, urgent requests and changes outside the map's domain are released at once. A new request similar to a pending one keeps the pending release time, so the timing is not reset every tick.

Gait-aware slew limiter. Once released, a change is applied through a slew limiter whose rate depends on phase. The limiter is built from the capability map: it ramps faster in phases where the map predicts little tilt for the current direction of change and slower where it predicts a lot. Its rate is scaled so that its mean over one period equals that of the uniform limiter, so it does not gain speed by being more aggressive on average.

Formation and queueing. Formations follow a reference point with slot offsets that shrink laterally when obstacles are close ahead. In bottlenecks, the robot further back takes 90% of the

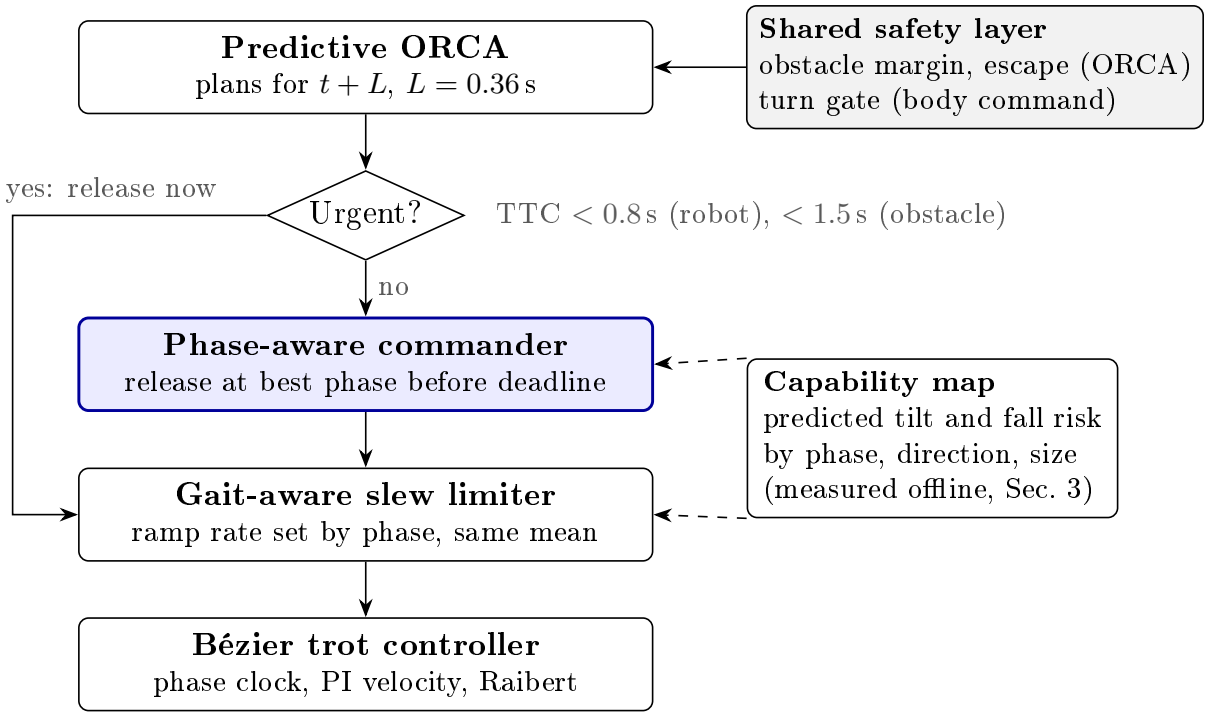


Figure 2: Per-robot stack. ORCA plans one gait period ahead; non-urgent changes wait in the commander for the phase the capability map predicts is safest, urgent ones go straight to the limiter. The safety layer is the same in every configuration.

avoidance responsibility in ORCA and the robot in front 10%, so that robots queue instead of shouldering through.

Shared safety layer. In every configuration, ORCA sees static obstacles inflated by 0.1 m with a 2 s horizon. Body commands are clipped to the range in which the gait was stress-tested ($|\omega_z| \le 1.0\,\mathrm{rad/s}$, $|v_y| \le 0.3\,\mathrm{m/s}$), and forward speed is scaled from 100% to 30% as the heading error grows from 30° to 90°, so a robot turns before it walks. If ORCA returns less than 0.05 m/s for more than 1 s while the robot wants to move, the preferred velocity is turned 75° towards the formation reference for 2 s, and to the other side if the robot is still stuck. Each of these was added after a failure found in the experiments; Section 7 reports one further fix that we tried and rejected.

5 Experimental setup

All experiments run in MuJoCo [15] with N copies of the Menagerie Go1 model [16] in one scene (foot friction 0.9). The swarm layer runs at 10 Hz and the gait controller at the physics rate. Each seed perturbs every start position by 5 cm (standard deviation) and gives every robot a random initial gait phase, so robots are never phase-locked. Table 1 lists the scenarios and Table 2 the configurations.

The main suite runs all six configurations on the 4- and 8-robot swaps, the 9-robot pillar field and the gate with 20 seeds (480 runs). The scaling suite runs the baseline, prediction only (with queueing) and the full system on the 16- and 32-robot swaps and the 16-robot pillar field with 10 seeds (90 runs). The lookahead suite varies $L \in \{0.18, 0.36, 0.54, 0.72\}$ s on the 8-robot

Table 1: Scenarios.

Scenario	Robots	Layout	Tests
Swap	4 / 8 / 16 / 32	On a circle of radius 2.5 / 4 / 6 / 9 m; each robot walks to the opposite point	Crossing traffic; density grows with N
Pillars	9 (3×3, 1.4 m spacing)	Formation travels 14 m at 0.35 m/s through 5 pillars ($r = 0.3$ m)	Formation through clutter
Gate	6 (2×3)	Wall with a 1.9 m opening	Bottleneck: the team narrows and queues
Pillars	16 (4×4)	18 m through 9 pillars	Dense formation through clutter
Course	1 and 6	53 m path at 0.45 m/s: pole slalom, 1.9 m gate, chicane, two obstacles moving across the path (0.4 m/s), 7 pillars	All of the above in sequence

Table 2: Configurations. All share the safety layer of Section 4.

Configuration	Prediction	Timing	Limiter	Queue
Baseline (reactive)	–	–	–	–
Queueing only	–	–	–	✓
Prediction only	✓	–	–	–
Prediction + timing	✓	✓	–	–
+ gait-aware limiter	✓	✓	✓	–
Full system	✓	✓	✓	✓

swap and the gate (10 seeds, 100 runs). The course runs three configurations for teams of 1 and 6 robots with 20 seeds (120 runs).

Metrics. Three metrics are fixed in advance as primary: urgent avoidance events per robot-minute (how often a robot had to change course with less than 0.8 s to spare), the 99th-percentile body tilt over the run, and the closest robot–robot distance. We also report falls (trunk below 0.15 m or tilted more than 60°), runs not finished within the time limit, contacts and completion time.

Statistics. Each configuration is compared with the baseline on the same seed. For each metric we take the per-seed difference and test it with a two-sided sign-flip permutation test (exact for up to 16 seeds, 20,000 random sign patterns otherwise), with 95% bootstrap confidence intervals. The 21 primary tests (7 scenarios $\times$ 3 metrics, full system against baseline) are corrected together with the Holm method. With 10 seeds the smallest attainable corrected p-value is about 0.02, which limits what the scaling suite can show.

6 Results

The full system reduces urgent events and tilt wherever robots crowd or obstacles force manoeuvres, and does no good in sparse traffic (Fig. 3, Table 3). Of the 21 primary tests, 15 are significant after Holm correction: 13 favour the full system and 2, both in the 4-robot swap, favour the baseline.

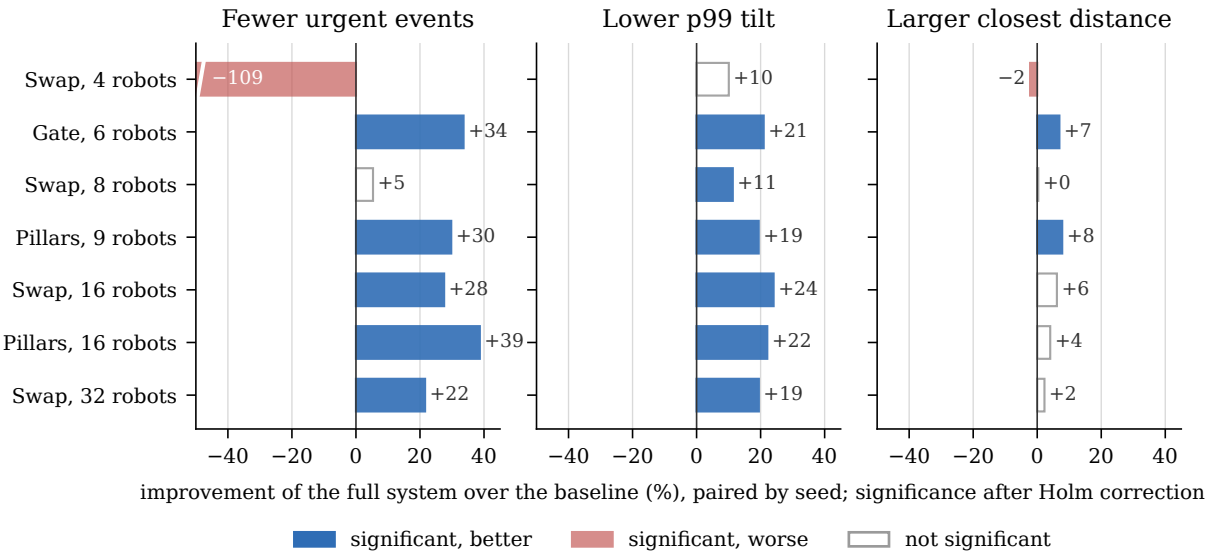


Figure 3: Improvement of the full system over the baseline on the three primary metrics. Filled bars are significant after Holm correction over all 21 tests. The 4-robot swap bar for urgent events is cut at −50% (true value −109%).

Table 3: Primary metrics, baseline (B) against full system (F), means over seeds. $^{*}p < 0.05$, $^{**}p < 0.01$ after Holm correction over all 21 tests.

			Urgent events / robot-min			p99 tilt (deg)			Closest distance (m)		
Scenario	N	Seeds	B	F	Change	B	F	Change	B	F	Change
Swap	4	20	5.2	10.9	+109%**	5.90	5.30	−10%	0.744	0.727	−2%**
Gate	6	20	24.4	16.2	−34%**	5.84	4.62	−21%**	0.661	0.707	+7%**
Swap	8	20	29.6	28.0	−5%	7.37	6.54	−11%**	0.626	0.628	+0%
Pillars	9	20	40.4	28.4	−30%**	7.14	5.76	−19%**	0.604	0.651	+8%**
Swap	16	10	54.1	39.2	−28%*	8.51	6.46	−24%*	0.600	0.637	+6%
Pillars	16	10	42.0	25.7	−39%*	7.22	5.62	−22%*	0.593	0.617	+4%
Swap	32	10	64.5	50.6	−22%*	8.39	6.76	−19%*	0.566	0.578	+2%

Which component does the work. Prediction alone does not help (Fig. 4). Against the baseline it changes urgent events by +6% in the 8-robot swap and +3% in the pillar field, and raises 99th-percentile tilt by 20% in the 4-robot swap. Adding phase timing produces most of the gain: urgent events fall by 15–16% and tilt by 12–16% in the 8-robot swap, the pillars and the gate. The gait-aware limiter adds further tilt reduction at the gate (−15% instead of −12%) but almost none elsewhere. Queueing matters in obstructed formations, where it doubles the reduction in urgent events (pillars −30% instead of −18%, gate −34% instead of −14%) and widens the closest distance by 7–8%. Phase timing is therefore the active ingredient, and queueing is a useful complement only where robots must pass through gaps.

Scaling. The benefit holds as the team grows. With 16 robots in a swap the full system cuts urgent events by 28% and tilt by 24%; with 32 robots, by 22% and 19%; with 16 robots in a formation through pillars, by 39% and 22%. The gain in closest distance shrinks at 32 robots (+2%, not significant), where the swarm is near its packing limit.

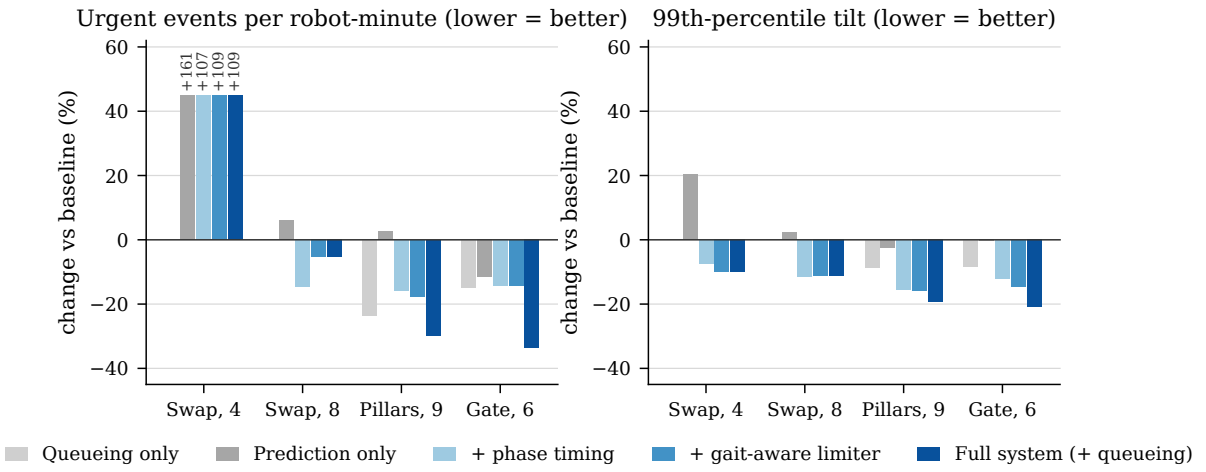
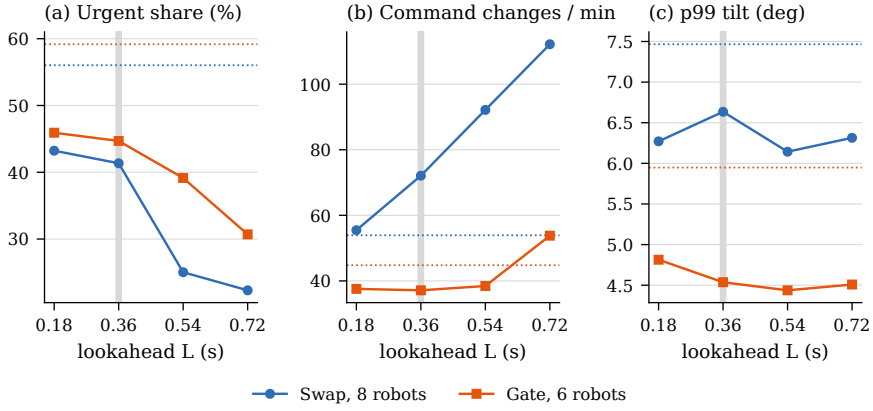


Figure 4: Ablation on the main suite (20 seeds). Change relative to the reactive baseline as components are added; lower is better. Prediction alone does not help; phase timing produces most of the gain.



Solid: full system at each lookahead; dotted: reactive baseline of the same colour. Grey band: L = 0.36 s (one gait period), used in all other experiments.

Figure 5: Effect of the lookahead L (10 seeds). Longer lookahead makes fewer manoeuvres urgent (a) but increases command churn (b); tilt is reduced about equally at every L (c).

Cost. Waiting for a good phase costs time when there is nothing to avoid. In the 4-robot swap the full system is 6.8% slower, has twice the urgent events of the baseline (5.2 against 10.9 per robot-minute) and a 2% smaller closest distance. In the 8-robot swap it is 2.7% slower; at the gate the completion time is unchanged.

Lookahead. Looking further ahead lets more changes be planned (Fig. 5): the share of manoeuvres that are urgent falls from 56% (baseline) to 43%, 41%, 25% and 22% for $L = 0.18$, 0.36, 0.54 and 0.72 s in the 8-robot swap. But the number of command changes per robot-minute doubles from 54 to 112, because predictions made further ahead are revised more often. Tilt is reduced about equally at every L. One gait period (0.36 s) is the setting we use; 0.54 s is slightly better at the gate but costs much more command churn in swaps.

Obstacle course. For the team of six, the full system lowered 99th-percentile tilt from 7.95° to 6.68° (−16%), the worst tilt over 20 runs from 18.3° to 13.7°, urgent events by 14% and robot–robot contacts from 0.25 to 0.05 per run, at the same completion time (129 s). Fig. 6

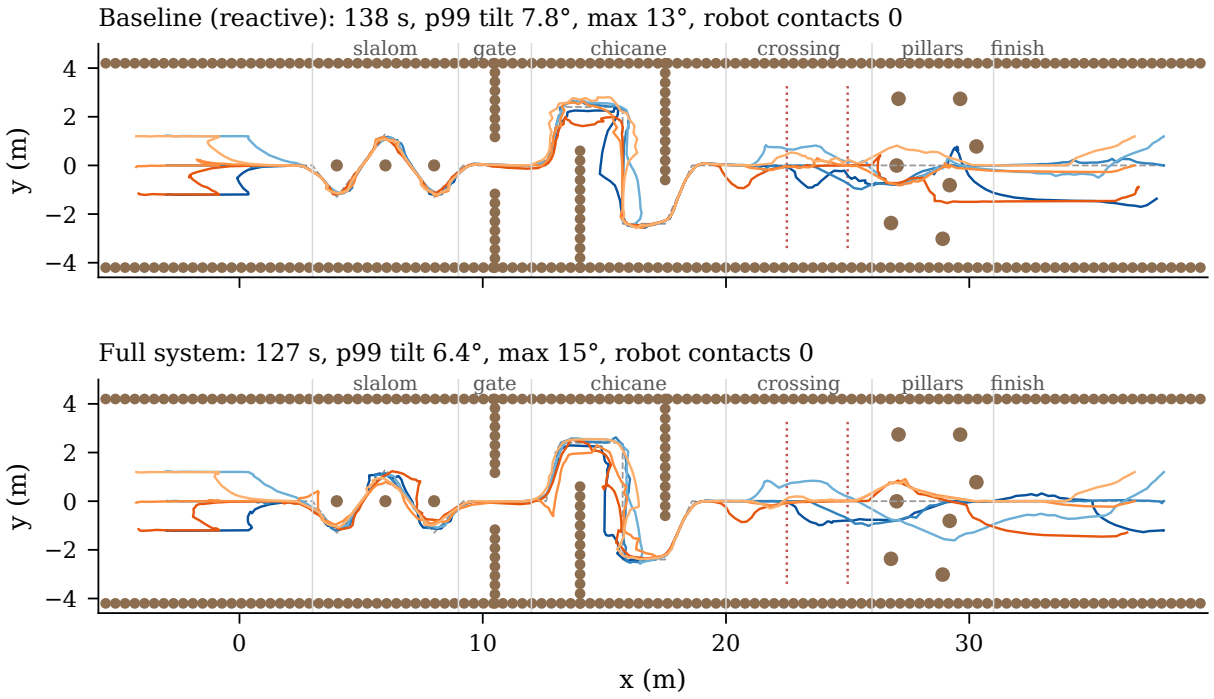


Figure 6: Paths of a six-robot team on the obstacle course (seed 0) for the baseline (top) and the full system (bottom). Dotted red lines: tracks of the two moving obstacles; dashed grey line: reference path.

shows one seed. One of its 20 runs did not finish in time; the baseline had one fall. For a single robot the three configurations are nearly identical (tilt 4.7–4.9°), and prediction raised urgent events (7.4 to 10.4 per minute), consistent with the 4-robot swap.

Safety totals. Across all 230 full-system runs in the final configuration there were no falls; the baseline fell twice in 170 runs (once in the 16-robot pillar field and once on the course). Runs that did not finish in time were rare in every configuration: one for the full system (course), one for queueing only and two for prediction only.

7 Discussion and limitations

When it helps. Phase timing pays off when a robot has many manoeuvres to make and enough warning to wait for a good phase. Crowds and clutter provide both; sparse traffic does not. In the 4-robot swap the few manoeuvres are mostly avoidable by acting slightly earlier, and any waiting only delays them, which shows up as more urgent events later. A practical system should enable timing only above a traffic-density threshold; we have not tested such a switch.

Robustness fixes and a rejected one. The shared safety layer was not designed in advance. Each part was added after a failure found by the experiments: robots falling after combined sideways walking and fast turning (turn gate), robots pinned against a wall between them and their slot (escape), and robots grazing pillars (static margin). Because the layer is identical in every configuration, it does not favour the full system. We also tried a stronger escape rule, which detects a stall by lack of progress instead of low speed and gives late robots right of way.

It removed every fall, but runs that did not finish in time rose from 1 to 16 of 480 in the main suite, because the rule also fired during normal avoidance and sent robots on long detours. We report it as a negative result; the code keeps it as an option.

Limitations. (i) All results are in simulation, with one robot model and one trot controller; the capability map must be re-measured for another controller, and the effect size on hardware is unknown. (ii) The capability map is built from single command changes in settled walking, and most swarm commands fall outside that domain, which is why prediction and a shaped limiter are needed alongside timing. (iii) Robots share positions and velocities perfectly, without noise or delay. (iv) The scaling suite has only 10 seeds, which limits the attainable significance. (v) Completion time rises by up to 7% in sparse scenarios.

8 Conclusion

A quadruped's tolerance of a velocity change depends on where in the gait cycle the change arrives, and a swarm layer can use this. Planning ORCA one gait period ahead and releasing each change at the predicted best phase cut urgent avoidance events by 22–39% and 99th-percentile tilt by 19–24% in crowded and obstructed scenarios with 6 to 32 simulated Go1 robots, with no falls in 230 runs. Prediction without timing gave no consistent benefit. Next steps are hardware experiments on Go1 robots, a second low-level controller (a learned policy) to test whether the capability map transfers, sensing noise and delay, and an automatic switch that disables timing in sparse traffic.

Declarations

Funding This research received no external funding.

Competing interests The author declares no competing interests.

Data and code availability The simulation code, configuration files, raw results and the scripts that produce every table and figure are held in a private repository and are available from the corresponding author on reasonable request. They will be made public on publication.

Author contributions Bishwaswarup Nayak conceived the study, developed the software, ran the experiments, analysed the results and wrote the manuscript.

Use of AI tools Claude (Anthropic) was used to assist with software development, experiment scripting, data analysis and drafting of the manuscript. The author reviewed and verified all code, results and text and takes full responsibility for the content.

Ethics approval Not applicable.

References

[1] Jur van den Berg, Ming Lin, and Dinesh Manocha. Reciprocal velocity obstacles for real-time multi-agent navigation. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 1928–1935, 2008.

[2] Jur van den Berg, Stephen J. Guy, Ming Lin, and Dinesh Manocha. Reciprocal n-body collision avoidance. In *Robotics Research (ISRR 2009)*, volume 70 of *Springer Tracts in Advanced Robotics*, pages 3–19. Springer, 2011.

[3] Jerry Pratt, John Carff, Sergey Drakunov, and Ambarish Goswami. Capture point: A step toward humanoid push recovery. In *Proc. IEEE-RAS International Conference on Humanoid Robots*, pages 200–207, 2006.

[4] Majid Khadiv, Alexander Herzog, S. Ali A. Moosavian, and Ludovic Righetti. Step timing adjustment: A step toward generating robust gaits. In *Proc. IEEE-RAS International Conference on Humanoid Robots*, 2016. arXiv:1610.02377.

[5] Hua Chen, Zejun Hong, Shunpeng Yang, Patrick M. Wensing, and Wei Zhang. Quadruped capturability and push recovery via a switched-systems characterization of dynamic balance, 2022. arXiv:2201.11928.

[6] Daan G. E. Hobbelen and Martijn Wisse. A disturbance rejection measure for limit cycle walkers: The gait sensitivity norm. *IEEE Transactions on Robotics*, 23(6):1213–1224, 2007.

[7] Weishu Zhan, Zheng Liang, Hongyu Song, and Wei Pan. Safe distributed learning-enhanced predictive control for multiple quadrupedal robots, 2025. arXiv:2503.05836.

[8] Magnus Norén, Marios-Nektarios Stamatopoulos, Avijit Banerjee, and George Nikolakopoulos. Role-adaptive collaborative formation planning for team of quadruped robots in cluttered environments, 2026. arXiv:2602.18260.

[9] Jur van den Berg, Jamie Snape, Stephen J. Guy, and Dinesh Manocha. Reciprocal collision avoidance with acceleration-velocity obstacles. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 3475–3482, 2011.

[10] Javier Alonso-Mora, Andreas Breitenmoser, Martin Rufli, Paul Beardsley, and Roland Siegwart. Optimal reciprocal collision avoidance for multiple non-holonomic robots. In *Distributed Autonomous Robotic Systems (DARS 2010)*, volume 83 of *Springer Tracts in Advanced Robotics*, pages 203–216. Springer, 2013.

[11] Yucheng Tao, Shaowen Cheng, Guorong Lan, Yanyan Yuan, Yongbin Jin, and Hongtao Wang. Extending the speed limit of quadrupedal locomotion via refined actuator modeling and adaptive command scheduling, 2026. arXiv:2609.13289.

[12] Dong Jin Hyun, Sangok Seok, Jongwoo Lee, and Sangbae Kim. High speed trot-running: Implementation of a hierarchical controller using proprioceptive impedance control on the MIT Cheetah. *The International Journal of Robotics Research*, 33(11):1417–1445, 2014.

[13] Maurice Rahme. spot_mini_mini: Dynamics and domain randomized gait modulation with Bezier curves for sim-to-real legged locomotion. GitHub repository, `https://github.com/OpenQuadruped/spot_mini_mini`, 2020. MIT licence.

[14] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, 1986.

[15] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.

[16] Kevin Zakka, Yuval Tassa, and MuJoCo Menagerie Contributors. MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo. GitHub repository, `https://github.com/google-deepmind/mujoco_menagerie`, 2022.